

Storefront Messaging — Technical PRD

Sep 23, 2026 · @Mann

Summary and scope

Customers can now hold a full two-pane conversation inbox on the storefront, at a theme page ending in `/messages`, without leaving the shop. The work spans three theme app extensions inside `extensions/hide-products`, all talking to the existing Shipturtle chat API through the Shopify app proxy.

Area	What changed	Files
Messages (new)	New Messages theme app block: conversation list, thread, composer, attachments, search, pagination, unread badges, realtime	<code>blocks/messages.liquid</code> , <code>assets/messages.js</code> (1,077 lines), <code>assets/messages.css</code> (1,002 lines)
Live Chat	Multi-file attachments ported over; login redirect fixed	<code>assets/live-chat.js</code> (+196), <code>assets/live-chat.css</code> (+171)
Seller Profile	"Chat with seller" button; Contact us tab hidden when live chat is on	<code>assets/seller-profile.js</code> (+40), <code>assets/seller-profile.css</code> (+20)
Shared	New i18n keys across 7 locales	<code>assets/lang.json</code> (+182)

Six API endpoints are consumed, all under the `/a/dashboard` proxy path. No token ever appears in theme code — the proxy injects it.

Out of scope: vendor-side inboxes (they live in the dashboard at

`C:\Users\ASUS\Desktop\shipturtle\frontendapp`) and any backend change (`/home/akanksha/shipturtle-api`). Every endpoint consumed here already existed.

Architecture and approach

The block is a single plain-JavaScript file that boots a Vue 3 app into a div the Liquid template renders. There is no build step, no bundler and no framework source in the repo — a theme app extension can only ship static assets, so every dependency arrives at runtime as a global.

flowchart TD

```

A["Merchant adds block<br/>to a /messages page"] --> B["messages.liquid"]
B --> C["Login gate<br/>customer == blank?"]
C -->|"not signed in"| D["Redirect to<br/>/account/login"]
C -->|"signed in"| E["div#st-messages-app<br/>+ window.stMessagesCustomer"]
E --> F["resources.js<br/>loadDependencies"]
F --> G["Vue, PrimeVue, vue-i18n,<br/>Aura, Pusher"]
G --> H["messages.js<br/>mounts the app"]
H --> I["/a/dashboard<br/>app proxy"]

```

The Liquid template passes data in through DOM attributes rather than inline script blobs: `data-lang-asset` points at `lang.json`, `data-block-settings` carries the merchant's settings as JSON. Customer identity is the one exception and rides on `window.stMessagesCustomer`.

Shared loader. `ST_Resources.loadDependencies(callback, extraDeps, featureName)` in `assets/resources.js` fetches Vue 3.5.1, PrimeVue, vue-i18n 9 and the Aura theme preset, preferring the Shopify asset CDN copies and falling back to unpkg. Messages passes Pusher 8.2.0 as an extra dependency. The same loader already served live-chat and seller-profile, so no new loading machinery was added.

Deliberate constraints. Vue templates are runtime-compiled template strings, so `app.config.compilerOptions.delimiters` stays `['{{', '}}']` and Liquid never sees them (the template lives in the JS asset, not the Liquid file). PrimeVue components are registered individually (`p-toast`, `p-file-upload`, `p-button`) rather than pulling the whole library in.

Style of the code. After an early draft grew an API abstraction layer, optimistic-send reconciliation and a cache, all of it was stripped back to match `live-chat.js`: flat `async` functions that fetch, map and assign to refs. The rule applied throughout was that a reader coming from live-chat should recognise every function.

Authentication and the app proxy

No API token exists anywhere in theme code. Every request goes to a relative path under `/a/dashboard`, which Shopify's app proxy forwards to the Shipturtle API after appending `shop`, `logged_in_customer_id`, `path_prefix`, `timestamp` and an HMAC `signature`. The backend trades those for the caller's identity.

On top of what the proxy adds, every call carries two query params of its own, matching what `live-chat.js` already sent:

```
const authParams = `shop=${shopDomain}&logged_in_customer_id=${customer.id || ''}`;
```

`shopDomain` comes from `window.Shopify.shop`; the customer id comes from the Liquid-rendered `window.stMessagesCustomer`.

Login gate. The page must not render for a signed-out visitor, so the check happens in Liquid, before any markup reaches the browser:

```
{%- if customer == blank and request.design_mode != true -%}
  <script>window.location.replace('/account/login');</script>
{%- else -%}
  ... the inbox ...
{%- endif -%}
```

Three details matter here:

- `request.design_mode` is true only inside the theme editor, where no customer is ever logged in. Without that guard the editor preview follows the redirect into a frame that cannot load `/account/login`, and the merchant sees a 404 while the live page works fine.
- A `<noscript>` link covers browsers with JavaScript disabled.
- The redirect is plain `/account/login` with **no** `return_url`. Return-after-login was tried and then deliberately dropped for Messages: the two customer-account systems take different parameters (`/account/login?return_url=` for classic accounts, `/customer_authentication/login?return_to=` for new customer accounts), and a wrong pairing silently drops the redirect. Live Chat keeps its own return-to behaviour because it redirects from a button press, not a page load.

API surface

Seven endpoints, all prefixed with `/a/dashboard` and all carrying `shop` and `logged_in_customer_id`.

Endpoint	Method	Extra params / body	Called when
<code>/customer/chats</code>	GET	<code>per_page=20</code> , <code>page</code> , <code>search</code>	Mount, search, scroll to bottom of list

Endpoint	Method	Extra params / body	Called when
<code>/customer/chats/{id}</code>	GET	—	Opening a conversation, after every send
<code>/customer/chats/{id}/messages</code>	POST	<code>{ message, attachments: [id] }</code>	Send
<code>/customer/chats/{id}/read</code>	POST	empty body	Opening a chat; incoming message
<code>/customer/chats/{id}/typing</code>	POST	empty body	Composer keystrokes, throttled
<code>/customer/chats/{id}/attachments</code>	POST	<code>FormData</code> with one file	Per selected file, before send
<code>/customer/chats/initiate</code>	POST	<code>{ initial_message, subject, vendor_id }</code>	First message to a new vendor
<code>/customer/chats/unread/count</code>	GET	—	Mount, and after each read

Mapped response keys. Only keys confirmed against real responses are read; nothing is guessed. The conversation row mapping is the whole contract for the sidebar:

```
const toConversation = (raw) => ({
  id: raw.id,
  vendorId: raw.vendor?.id ?? raw.vendor_id ?? null,
  isVendorLevel: !raw.queryable,
  productImage: raw.queryable?.image || '',
  productTitle: raw.queryable?.title || raw.vendor?.brand_name || '',
  vendorName: raw.vendor?.brand_name || raw.queryable.vendor || '',
  lastMessage: raw.latest_message?.message || '',
  unreadCount: Number(raw.unread_count) || 0
});
```

Source key	Shown as
<code>data.data[]</code>	The chat list (Laravel paginator, so rows are nested one level deep)
<code>queryable.title</code>	Conversation title; falls back to the brand name on a vendor-level chat
<code>queryable.image</code>	Conversation thumbnail
<code>vendor.brand_name</code>	Vendor line under the title
<code>latest_message.message</code>	Preview text
<code>unread_count</code>	Per-row badge
<code>data.unread_count</code> (on <code>/unread/count</code>)	Inbox heading badge
<code>attachments[].url</code>	Rendered attachment in a bubble
<code>sender_type === 'customer'</code>	Which side of the thread a bubble sits on
<code>read_by[]</code>	Seen acknowledgement

Pagination follows the Laravel paginator: `current_page` and `last_page` off `data`, rows off `data.data`.

Error handling is centralised in `fetchViaAppProxy`. A 404 or 5xx also notifies the developer through `ST_Resources.notifyDeveloper`. `FormData` bodies deliberately send no `Content-Type` so the browser can set the multipart boundary, and an empty POST response body is treated as success rather than a parse failure.

Messages block: UI and state

Two panes side by side, collapsing to one pane on mobile where `is-thread-open` swaps which is visible.

```

flowchart LR
  R["st-messages-root"] --> L["List pane"]
  R --> T["Thread pane"]
  L --> L1["Inbox heading<br/>+ unread badge"]
  L --> L2["Search field"]
  L --> L3["Conversation rows<br/>infinite scroll"]
  T --> T1["Thread header<br/>title + vendor"]
  T --> T2["Message bubbles<br/>+ attachments"]
  T --> T3["Typing indicator"]
  T --> T4["Composer: chips,<br/>paperclip, send"]

```

The left pane header was built to a supplied design: an "Inbox" heading with a leading envelope icon and a search field below it. The All / Unread / Archived filter row was removed, and no "Messages" heading is rendered. With no conversation selected the thread pane shows a placeholder rather than an empty frame.

State. Everything is flat refs on the setup function; there is no store.

Ref	Holds
<code>conversations</code>	Mapped sidebar rows, appended across pages
<code>threadMessages</code>	Messages of the open chat
<code>activeConversationId</code> , <code>chatId</code>	Selection
<code>draftConversation</code>	A vendor thread that has no chat row yet
<code>attachments</code> , <code>uploadedIds</code>	Pending uploads and the ids to send
<code>searchTerm</code> , <code>currentPage</code> , <code>lastPage</code>	List querying
<code>isLoadingConversations</code> , <code>isLoadingMore</code> , <code>isLoadingMessages</code> , <code>isSending</code>	The four loading states
<code>isSellerTyping</code> , <code>totalUnread</code> , <code>isThreadOpen</code>	Presentation

Loading states are distinct on purpose. The list renders seven skeleton rows on first load; the thread renders skeleton bubbles sized to real bubbles (an earlier version used oversized blocks and looked wrong); the send button uses PrimeVue's `loading` prop and is disabled while in flight; paging appends without disturbing the rows already on screen. A refetch after send passes `showLoader: false` so the thread does not blank out between the POST and the refresh.

Core flows

Page load

```
sequenceDiagram
    participant B as Browser
    participant P as App proxy
    participant A as API
    B->>P: GET /customer/chats?per_page=20&page=1
    P->>A: forwarded + token
    A-->>B: data.data[] + paginator
    B->>B: map rows, hide skeletons
    B->>P: GET /customer/chats/unread/count
    A-->>B: data.unread_count
    B->>B: heading badge
```

The unread count is not awaited — the list must not wait on a badge. If the page was opened as `?vendor=<id>` from a seller profile, the linked vendor is resolved between those two steps.

Search and pagination

Both are server-side. Typing debounces 350 ms then refetches page 1 with `search=<term>`; results replace the list. Scrolling within 90 px of the bottom requests the next page and appends, guarded by `currentPage < lastPage` and the `isLoadingMore` flag so one scroll cannot fire two requests.

Opening a conversation

```
sequenceDiagram
    participant U as Customer
    participant V as Vue app
    participant A as API
    U->>V: click a row
    V->>V: clear draft, show skeleton bubbles
    V->>A: GET /customer/chats/{id}
    A-->>V: messages[] with attachments, read_by
    V->>V: render, scroll to bottom
    V->>A: subscribe private-support-chat.{id}
    V->>A: POST /customer/chats/{id}/read
    V->>A: GET /customer/chats/unread/count
```

The per-row badge is zeroed locally the instant the chat opens, because the list is not refetched; only the heading total waits on the server round trip.

Sending a message

```
sequenceDiagram
    participant U as Customer
    participant V as Vue app
    participant A as API
    participant Pu as Pusher
    U->>V: type, attach, press Send
    V->>V: isSending = true, button disabled
    V->>A: POST /customer/chats/{id}/messages<br/>{message, attachments:[ids]}
    A->>Pu: broadcast message.sent
    V->>A: GET /customer/chats/{id} (no loader)
    A-->>V: refreshed thread
    V->>V: clear draft + chips, scroll down
    Pu-->>V: message.sent echo of our own message
```

That last line is the one known wart: the customer is subscribed to the same channel they publish on, so their own message comes back. See Open items.

When the thread has no `chatId` yet — a vendor opened from a seller profile who has never been written to — the send is routed to `/customer/chats/initiate` instead, with `initial_message`, a `subject` of `Customer<>Brand`, and `vendor_id`. The list is then refetched, the new chat located and opened.

Attachments

The flow mirrors the dashboard's, in `src/store/liveChatStore.ts` and `ChatWindow.vue` of the frontend repo: **upload first, send the ids second**. The attachment endpoint takes one file per request, so a multi-file selection fans out into parallel uploads.

```
flowchart TD
    A["Customer picks files<br/>via paperclip"] --> B["One chip per file,<br/>status = uploading"]
    B --> C["POST ../attachments<br/>one request per file"]
    C -->|"ok"| D["chip status = success,<br/>store returned id"]
    C -->|"fails"| E["chip status = failed,<br/>error toast"]
    D --> F["Send enabled"]
    F --> G["POST ../messages<br/>{message, attachments:[ids]}"]
    E --> H["Customer removes<br/>the chip"]
```

Chips sit above the text field with a remove control on each, exactly as the dashboard does it. `uploadedIds` is a computed over the successful chips, so failed uploads simply never reach the send body. Send stays enabled when there is text **or** at least one uploaded file, which is how attachment-only messages are possible.

Picker. PrimeVue `FileUpload` in basic mode supplies the paperclip, with `multiple` on and:

```
accept: '.jpg,.jpeg,.png,.gif,.webp,.pdf,.doc,.docx,.xls,.xlsx,.txt,.csv'
```

Styling it took a compound selector. PrimeVue builds its choose button as `chooseButtonClass: [cx('pcChooseButton'), this.class]`, which puts our class on the button rather than on an ancestor — so `.st-msg-attach .p-button` matched nothing and Aura's emerald primary showed through. `.st-msg-attach.p-button`, plus `:focus` and `:focus-visible` overrides, fixed the green button and its focus ring.

Rendering. Incoming attachments come back on `attachments[].url` and are rendered as-is, with no formatting or filename derivation. One layout trap is worth recording: an image with `width: 100%` inside a shrink-to-fit flex bubble collapses to nothing when the message text is empty, which is exactly the attachment-only case. Fixed by giving the image a fixed base width (`260px` in Messages, `220px` in Live Chat) with `max-width: 100%`.

Attachments are disabled on a draft thread in both extensions, since uploading needs a `chatId` that does not exist until the chat is initiated.

Realtime

Pusher is initialised once in `onBeforeMount` on cluster `ap2`, authenticating private channels through the proxy at `/customer/chats/broadcasting/auth`. Messages and Live Chat currently share the same app key, `d7db7850adabc9323922`, which is the development key and must not be swapped.

Opening a chat subscribes to `private-support-chat.{chatId}`; switching chats unsubscribes first, so only one channel is ever live.

Event	Handler does
<code>message.sent</code>	Skips if the id is already in the thread, else appends, updates the sidebar preview, scrolls down, marks read
<code>message.read</code>	Appends <code>{participant_id, participant_type, read_at}</code> to each message's <code>read_by</code> , driving the seen tick
<code>user.typing</code>	Ignores events where <code>sender_type === 'customer'</code> , else shows the indicator for 2 s

Echo semantics are the thing to understand here. Pusher delivers a broadcast to every subscriber of the channel, including the client whose own action caused it. The customer publishes and subscribes on the same channel, so their own `message.sent` and `user.typing` come back to them.

The typing handler already accounts for this with `if (data.sender_type === 'customer') return`. The `message.sent` handler does not, which is why sending a message triggers a read receipt for that message — covered in Open items.

Outgoing typing is throttled by `lastTypingSentAt`, so keystrokes do not turn into a request per character.

Unread counts

Two badges, two sources.

Badge	Source	Refreshed
Per conversation row	<code>unread_count</code> on each chat in <code>/customer/chats</code>	Every list fetch; zeroed locally on open
Inbox heading total	<code>data.unread_count</code> from <code>/customer/chats/unread/count</code>	On mount, and after every successful read

The heading badge caps display at `99+`. Both hide at zero rather than showing a `0`.

The refresh call sits **inside** `markChatAsRead`, after the read POST resolves, rather than at the call site:

```
await fetchViaAppProxy(`/customer/chats/${chatId.value}/read?${authParams}`,
  { method: 'POST' });
// This chat's unreads just went to zero, so the heading total is stale.
await fetchUnreadCount();
```

Putting it there means both paths that clear unreads — opening a chat, and a seller's message arriving while the chat is open — keep the heading accurate without duplicating the call. It runs after the read resolves so the backend has already zeroed the chat before the new total is requested.

The row badge is cleared optimistically on the client because the conversation list is deliberately not refetched when a chat is opened; waiting for the server would leave a stale badge on the row the customer is reading.

Live Chat extension changes

The attachment work built for Messages was ported back so the two surfaces behave the same. Nothing about the existing chat flow changed.

Change	Detail
Attachments	Same upload-then-send model: chips above the composer, one POST per file, ids on the send body
Rendering	Attachments render inside message bubbles from <code>attachments[].url</code>
Refetch after send	The thread refreshes after a successful send, matching Messages
Login redirect	Now <code>/customer_authentication/login?return_to=<path></code> for the new customer accounts system

Two CSS namespaces were added: `.st-chat-attachments` / `.st-chat-chip*` for the composer chips, and `.st-chat-atts` / `.st-chat-att-*` for the in-bubble rendering. As in Messages, the choose button needed the compound `.st-chat-attach.p-button` selector, here with `!important`, to escape Aura's primary colour.

One practical note for anyone editing this file: `live-chat.js` is uniformly CRLF, so any script that rewrites it has to normalise line endings or the edit silently fails to match.

Seller Profile extension changes

A "Chat with seller" button now sits in the vendor details header, and the Contact us tab steps aside when live chat is on. Both hang off the same flag, `parent_company.live_chat_enabled`.

```

flowchart LR
    A["live_chat_enabled?"] -->|"true"| B["Show Chat with seller"]
    A -->|"true"| C["Hide Contact us tab"]
    A -->|"false"| D["Hide button,<br/>keep Contact us tab"]
    B --> E["/pages/messages<br/>?vendor=&vendor_name="]
```

The tab gate is one clause on the existing `hasTab` switch:

```

case 'contact': return !!settings.seller_contact_us_tab &&
!parentCompany.value.live_chat_enabled
```

The button navigates rather than opening a chat in place, because the conversation belongs on the Messages page:

```
const params = new URLSearchParams({
  vendor: vendor.id,
  vendor_name: vendor.brand_name || vendor.title || ''
})
window.location.href = `/pages/messages?${params.toString()}`
```

The name rides along so Messages can title a brand-new thread before any chat with that vendor exists. If the vendor has no id the button falls back to `/pages/messages` with no params. Sign-in is not handled here — the Messages page's own gate takes care of it.

This is a vendor-level conversation with no product attached, so the Messages side renders no product thumbnail and no "Sold by" line for it. Button sizing, padding and font colour were tuned to match the supplied design.

Localization, settings and theming

Localization lives in `assets/lang.json`, not in `locales/*.json`. This is the convention the other theme app extensions follow — seller-profile and write-a-review both read a single JSON asset at runtime through `data-lang-asset` and hand it to `vue-i18n`. An early draft put keys in `locales/de.json`, `locales/en.json` and so on; that was reverted.

All seven locales are complete: `en`, `de`, `es`, `nl`, `pt`, `no`, `ro`, with 20 keys under `messages` and 13 under `live-chat`. Locale selection reads `document.documentElement.lang`, then `window.Shopify.locale`, then the browser, falling back to `en`. Dates render with `en-GB` for English so headers read day-first.

Block settings are deliberately few:

Setting	Purpose
<code>panel_height</code>	Any CSS length; defaults to <code>640px</code>
<code>composer_placeholder</code>	Message box placeholder
<code>quick_replies</code>	Comma-separated chips above the composer; blank hides them
<code>customCss</code>	Liquid-type setting for merchant CSS, documented with example selectors

Theming runs on CSS custom properties declared on `.st-messages-root`, which is also what `customCss` is told to target. Native scrollbars were an unexpected problem: they inherit the page's `color-scheme`, so on a dark-ish theme they render black. Rather than hardcode a colour or force `color-scheme: light`, `::-webkit-scrollbar` and `scrollbar-color` are styled from the same panel tokens, so a merchant retheming the panel gets matching scrollbars for free.

Focus styling is deliberately untouched: the composer keeps whatever focus treatment the theme provides, with no override of its own.

Open items

Read receipt fires on your own send

Status: diagnosed, fix not yet applied. Sending a message produces a `POST /read` and a `GET /unread/count` it does not need. `sendMessage` calls neither; the `message.sent` Pusher handler does, on the echo of the customer's own message. The dedupe guard above it does not catch this because the thread refetch happens after the POST, so the echo usually arrives before the id is in the array.

The fix is the filter the typing handler already uses:

```
if (data.sender_type !== 'customer') markChatAsRead();
```

"Authentication required" on the live storefront

Status: unresolved, needs a captured request. The proxy answers `{"success": false, "message": "Authentication required. Please provide a valid JWT or access via Shopify store."}` on `/pages/messages` in production. Two candidates, neither confirmed:

- Messages hardcodes `baseApiUrl = '/a/dashboard'`, while Live Chat reads a configurable `app_proxy_prefix` block setting. A store whose proxy is mounted elsewhere would fail exactly this way.
- Both our `shop / logged_in_customer_id` and Shopify's own appended copies reach the backend, and the duplicates may break HMAC signature verification.

The failing request URL from the Network tab settles which.

Typing indicator does not appear in Messages

Status: client side ruled out. Markup, state, bindings, channel name, throttle and CSS were all compared against Live Chat and are equivalent, and both extensions now use the same Pusher key on the same cluster. That points at the backend not broadcasting `user.typing` for this path rather than at the theme code.

Confirmations still needed

Item	Where
Which key carries the vendor id on a chat row	<code>toConversation</code>
The key <code>/customer/chats/initiate</code> expects for a vendor-level chat	<code>startChatWithVendor</code>
Which identifier the Messages page should key a vendor thread on	<code>chatWithSeller</code> in seller-profile

Accepted limitations

- Attachments are disabled on a draft thread in both extensions, because uploading requires a `chatId` that does not exist until the chat is initiated.
- The heading unread total does not decrement on its own as a seller's messages arrive; it refreshes on mount and after each read.

Change log

All paths relative to `extensions/hide-products/`.

File	Status	Contents
<code>blocks/messages.liquid</code>	New, 99 lines	Login gate, asset loading, customer globals, block schema with 4 settings plus custom CSS
<code>assets/messages.js</code>	New, 1,077 lines	The whole inbox: fetching, mapping, thread, composer, attachments, Pusher, i18n
<code>assets/messages.css</code>	New, 1,002 lines	Panel tokens, two-pane layout, bubbles, chips, skeletons, badges, scrollbars
<code>assets/live-chat.js</code>	+196	Attachments, refetch after send, login redirect

File	Status	Contents
assets/live-chat.css	+171	Composer chips and in-bubble attachment rendering
assets/seller-profile.js	+40	chatWithSeller , Contact us tab gate
assets/seller-profile.css	+20	Chat with seller button
assets/lang.json	+182	messages and live-chat sections across 7 locales

shopify.app.toml also shows a one-line change unrelated to this work.

Related repos. The dashboard implementation the attachment flow was modelled on is at C:\Users\ASUS\Desktop\shipturtle\frontendapp (src/store/liveChatStore.ts , ChatWindow.vue). The API is at /home/akanksha/shipturtle-api ; no backend change was needed — every endpoint consumed already existed.